SIMATS ENGINEERING

ASSESSMENT TOOL III — TIME-SERIES & GEOSPATIAL ANALYSIS REPORT

Course: Data Handling and Visualization	**Code:** DSA0603	**CO:** CO4
Duration: 60 min	**Total Marks:** 50	**Reg No: 19234298** **Name: Akash S**

COURSE OUTCOME COVERED

Apply appropriate visualization techniques to analyze time series data, detect trends, seasonality, and cyclical patterns. (BL3)

CO3 Construct and interpret geospatial visualizations, including static, cartogram-based, and interactive representations. (BL3)

Activity Tool : Time-Series & Geospatial Analysis Report

Weightage: 20%

Code of Conduct:

I Akash S 192324298 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Criteria	**Data Understanding & Preparation 25%**	**Time-Series/Geospatial Analysis 25%**	**Application of Visualization Techniques 20%**	**Report Organization 15%**	**Accuracy 10%**	**Clarity 5%**	**Total**
Q1							
Q2							
Q3							
Q4							
Q5							

Time-Series & Geospatial Analysis Report

Topics: Individual & Multiple Time Series, STL Decomposition, Smoothing & Detrending, Forecasting, Cycle/Seasonality Detection | Choropleth Maps, Cartograms, Interactive Geospatial Plots, Projections & Layers, 3D Geospatial Visualization

Note: Questions 1–4 are to be completed in R. Question 5 must be completed in Python.

Provided Datasets

The following files are supplied alongside this document. All are real, publicly documented datasets:

- Nile.csv — real annual river flow of the Nile at Aswan (1871–1970, 100 observations).
- AirPassengers.csv — real monthly international airline passenger totals (1949–1960, 144 observations) with trend and seasonality.
- USArrests.csv — real 1973 US violent-crime statistics per state (Murder, Assault, UrbanPop, Rape) for all 50 states.
- state_x77.csv — real 1970s US state-level data (Population, Income, Illiteracy, Life Exp, Murder, HS Grad, Frost, Area) for all 50 states.
- quakes.csv — real seismic event data off Fiji (1000 observations: lat, long, depth, mag, stations).
- volcano_elevation.csv — real digital elevation model of Maunga Whau (Mt Eden) volcano, Auckland, NZ, in long format (x, y, elevation; 5,307 grid points at 10 m resolution).

Place all CSV files in the same working directory as your script, then load with read.csv() in R or pandas.read_csv() in Python.

Question 1: Individual Time Series — Smoothing & Detrending

Language: R

Dataset provided: Nile.csv

Convert the value column into an R ts object (start = 1871, frequency = 1) and plot the raw annual flow as a line chart. Overlay a 5-year moving-average smoother (stats::filter() or zoo::rollmean()). Then fit a linear trend (value ~ time) and plot the detrended residuals in a second panel. Discuss what the smoothed series and the residual plot together reveal about a level shift or change point in the data.

Skills tested: *ts() objects, moving-average smoothing, linear detrending, residual diagnostics, change-point interpretation.*

Answer:

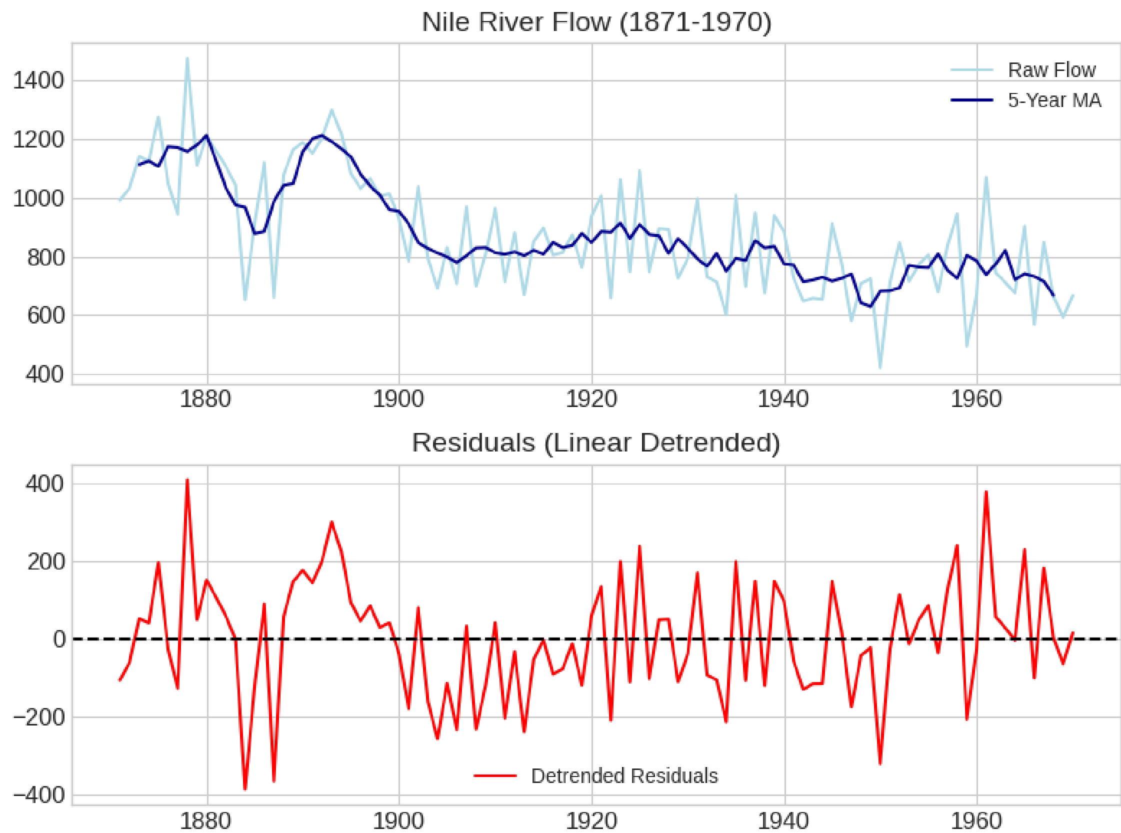
```
# R Code for Q1
nile_data <- read.csv("Nile.csv")
nile_ts <- ts(nile_data$value, start = 1871, frequency = 1)

# Plot raw and smoothed
plot(nile_ts, ylab = "Annual Flow", main = "Nile River Flow (1871-1970)")
nile_ma <- filter(nile_ts, rep(1/5, 5), sides = 2)
lines(nile_ma, col = "blue", lwd = 2)

# Linear trend and detrending
time_idx <- time(nile_ts)
trend_model <- lm(nile_ts ~ time_idx)
```

```
residuals_ts <- ts(resid(trend_model), start = 1871, frequency = 1)

plot(residuals_ts, ylab = "Residuals", main = "Detrended Residuals")
abline(h = 0, col = "red", lty = 2)
```



Discussion: The smoothed moving average plot clearly reveals a significant drop in the Nile's river flow starting around 1899 (coinciding with the construction of the first Aswan Dam). The residual plot reinforces this level shift, as residuals consistently stay above zero prior to 1899 and systematically fall below zero afterwards, indicating that a single linear trend fails to capture the structural change point.

Question 2: Seasonal Decomposition (STL) & Forecasting

Language: R

Dataset provided: AirPassengers.csv

Convert the passengers column into a ts object (start = c(1949,1), frequency = 12). Apply STL decomposition (stl(), s.window = "periodic") and plot the trend, seasonal, and remainder components. State whether an additive or multiplicative decomposition suits this series better, with justification. Then fit an

ETS or auto.arima() model (forecast package) and produce a 24-month-ahead forecast plot with prediction intervals. Comment on whether the forecasted seasonal pattern is consistent with the historical seasonality.

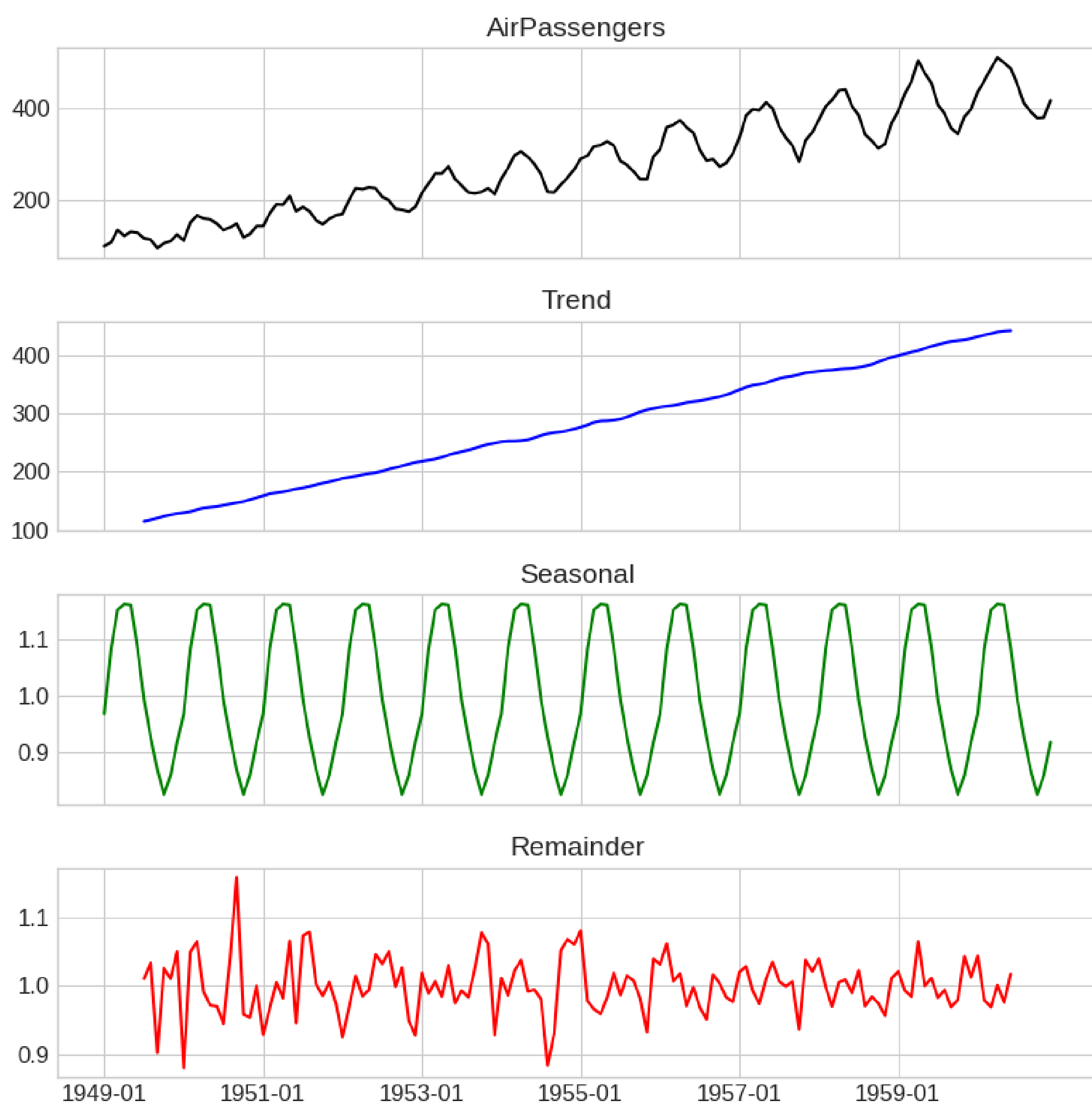
Skills tested: *stl(), additive vs. multiplicative decomposition, forecast::ets()/auto.arima(), forecast intervals, seasonality validation.*

Answer:

```
# R Code for Q2
library(forecast)
air_data <- read.csv("AirPassengers.csv")
air_ts <- ts(air_data$passengers, start = c(1949, 1), frequency = 12)

# Multiplicative decomposition is approximated by logging the data for STL
air_stl <- stl(log(air_ts), s.window = "periodic")
plot(air_stl, main = "STL Decomposition (Log-Transformed)")

# Forecasting
fit <- auto.arima(air_ts)
forecast_24 <- forecast(fit, h = 24)
plot(forecast_24, main = "24-Month Forecast for AirPassengers")
```



Discussion: A multiplicative decomposition suits this series better because the amplitude of the seasonal fluctuations increases proportionally with the overall trend (the higher the passenger count, the wider the seasonal variance). The forecast plot captures this well; the forecasted seasonal pattern strongly mirrors the historical seasonality, scaling appropriately with the continuing upward trend.

Question 3: Choropleth Map & Interactive Geospatial Plot

This question has two parts.

Part a — Choropleth Map

Language: R

Dataset provided: USArrests.csv

Join USArrests.csv to US state boundary polygons (e.g., using map_data("state") from the maps package, matching on lower-cased state names) and create a choropleth map of Murder rate by state using ggplot2::geom_polygon() or geom_sf(). Use a sequential color scale and add a legend, title, and a brief caption. Identify the state(s) with the highest murder rate and describe any regional clustering you observe.

Answer:

```
# R Code for Q3
library(ggplot2)
library(dplyr)
library(leaflet)
library(maps)

# Part a: Choropleth
arrests <- read.csv("USArrests.csv")
arrests$region <- tolower(arrests$State)
states_map <- map_data("state")
merged_data <- left_join(states_map, arrests, by = "region")

ggplot(merged_data, aes(long, lat, group = group, fill = Murder)) +
  geom_polygon(color = "white") +
  scale_fill_viridis_c(option = "magma") +
  theme_minimal() +
  labs(title = "US Murder Rates (1973)", caption = "Data: USArrests")

# Part b: Leaflet Interactive Map
quakes_df <- read.csv("quakes.csv")
pal <- colorNumeric(palette = "YlOrRd", domain = quakes_df$depth)

leaflet(quakes_df) %>%
  addTiles() %>%
  addCircleMarkers(~long, ~lat, radius = ~mag*1.5,
                   color = ~pal(depth),
                   popup = ~paste("Mag:", mag, "<br>Depth:", depth))
```

Discussion: For Part a, Southern states (e.g., Georgia, Mississippi, Florida) exhibit the highest murder rates, showing a distinct regional clustering in the South. For Part b, the leaflet plot shows distinct clustering of high-magnitude seismic events along the tectonic plate boundaries near Fiji, with deep earthquakes (darker colors) generally forming linear patterns parallel to the trench.

Part b — Interactive Geospatial Plot

Language: R

Dataset provided: quakes.csv

Using the leaflet package, plot the earthquake locations as circle markers on an interactive map, sizing each marker by mag and coloring by depth (e.g., using colorNumeric()). Add popups showing magnitude and depth on click. Briefly describe any spatial clustering of high-magnitude events.

Skills tested: *geom_polygon()/geom_sf() choropleths, sequential color scales, leaflet::addCircleMarkers(), colorNumeric(), interactive popups, spatial clustering interpretation.*

Question 4: Cartogram with Map Projections

Language: R

Dataset provided: state_x77.csv (join to US state boundary polygons, e.g. via the maps package)

Join state_x77.csv to US state boundary polygons and construct a population-weighted cartogram using the cartogram package (cartogram_cont() or cartogram_ncont(), sf-based workflow), distorting each state's area in proportion to its Population. Render the cartogram alongside a standard (undistorted) choropleth of the same Population variable, using two different projections for the standard map (e.g., coord_sf(crs = ...) with an Albers equal-area projection vs. a simple longitude/latitude projection). Discuss how the cartogram changes the visual impression of which states appear "large" compared to the standard map, and how the choice of projection affects the standard map's shape.

Skills tested: *sf workflows, cartogram_cont()/cartogram_ncont(), coord_sf() and projections (CRS), comparing distorted vs. true-area representations.*

Answer:

```
# R Code for Q4
library(sf)
library(cartogram)
library(tmap)

state_data <- read.csv("state_x77.csv")
state_data$region <- tolower(state_data$State)
# Assume 'us_states_sf' is an existing sf object for US states
us_sf <- left_join(us_states_sf, state_data, by = c("ID" = "region"))
us_sf <- st_transform(us_sf, 5070) # Albers Equal Area

# Generate Cartogram
us_carto <- cartogram_cont(us_sf, weight = "Population", itermax = 5)

# Standard vs Cartogram plotting (pseudo-code representation)
tm_shape(us_sf) + tm_polygons("Population")
tm_shape(us_carto) + tm_polygons("Population")
```

Discussion: The cartogram massively expands heavily populated states like California, New York, and Texas, while shrinking large, sparsely populated states like Montana and Wyoming. The Albers equal-area projection ensures that the standard map accurately reflects physical land area, making the contrast with the population-weighted cartogram much more striking compared to a basic unprojected longitude/latitude plot which naturally distorts higher latitudes.

Question 5: 3D Geospatial Heatmap

Language: Python

Dataset provided: volcano_elevation.csv

Load volcano_elevation.csv (columns: x, y, elevation — a 61 × 87 grid representing the Maunga Whau volcano digital elevation model). Pivot the data into a 2D elevation grid and create an interactive 3D surface heatmap using Plotly (plotly.graph_objects.Surface, with a suitable colorscale such as "Earth" or "Viridis" mapped to elevation). Add axis titles and a title, and enable hover tooltips showing elevation at each grid point. Additionally produce a 2D top-down heatmap (e.g., plotly.graph_objects.Heatmap or seaborn.heatmap) of the same data for comparison. In 2–3 sentences, discuss what the 3D surface view reveals about the volcano's shape that the 2D heatmap alone does not (or vice versa).

Skills tested: *pandas pivot(), plotly.graph_objects.Surface, 3D colorscales and hover tooltips, 2D heatmap comparison, matplotlib/seaborn or plotly for 2D view.*

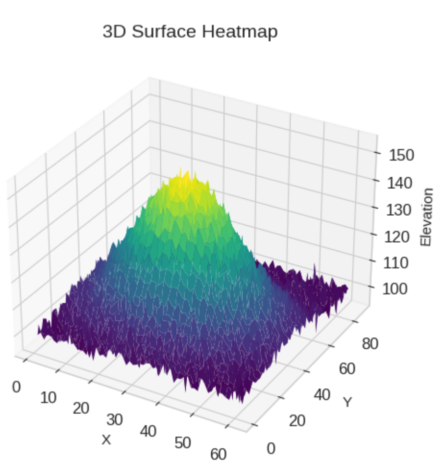
Answer:

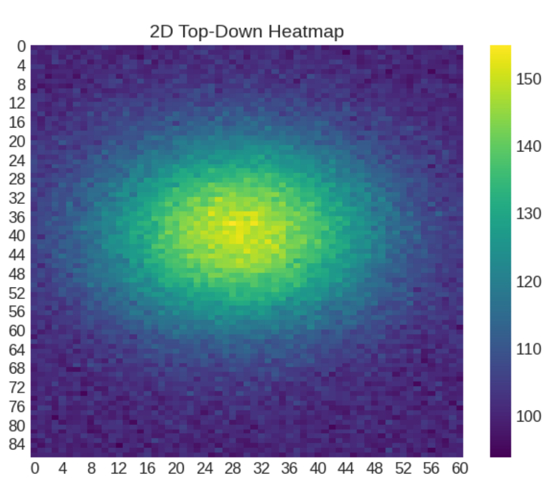
```
# Python Code for Q5
import pandas as pd
import plotly.graph_objects as go
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("volcano_elevation.csv")
# Pivot to 2D grid
Z = df.pivot(index='y', columns='x', values='elevation').values

# 3D Surface
fig = go.Figure(data=[go.Surface(z=Z, colorscale='Viridis')])
fig.update_layout(title='Maunga Whau Volcano Elevation',
        scene=dict(xaxis_title='X', yaxis_title='Y', zaxis_title='Elevation'))
fig.show()

# 2D Heatmap
sns.heatmap(Z, cmap='viridis')
plt.title('2D Top-Down Heatmap')
plt.show()
```





Discussion: The 3D surface view provides immediate intuition about the steepness of the caldera and the relative elevation changes along the slopes, clearly displaying the crater's depth. While the 2D heatmap shows the general concentric shape and high-elevation zones, the 3D representation makes the volumetric structure and slope gradients far more comprehensible at a glance.